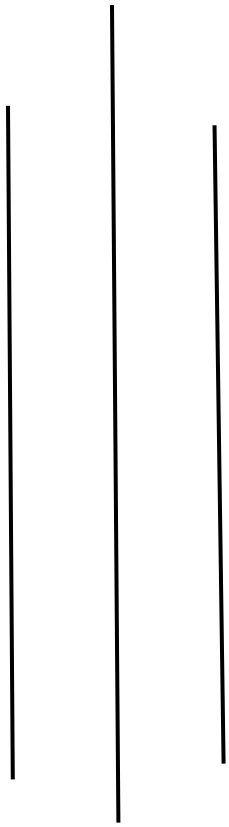
WEEK 3: INTERNSHIP REPORT



Submitted By: Yanji Pakhrin

To

OneCover Nepal

TABLE OF CONTENTS

CHAPTER 1: WEB APPLICATION SECURITY 1

1.1 Introduction 1

1.2 Objective 1

CHAPTER 2: Overview of Security Concepts 2

2.1 SQL Injection 2

2.2 Cross-Site Scripting (XSS) 3

2.3 Authentication 4

CHAPTER 3: Practical Task 5

3.1 Lab 1: SQL Injection (Hidden Data) 5

3.2 Lab 2: SQL Injection (Login Bypass) 10

3.3 Lab 3: SQL Injection (Union Attack) 15

3.4 Lab 4: Cross-Site Scripting (Reflected XSS) 18

3.5 Lab 5: Cross-Site Scripting (Stored XSS) 21

3.6 Lab 6: User Enumeration 24

3.7 Lab 7: 2FA Simple Bypass 30

3.8 Challenges 33

3.9 Key Learnings 33

3.10 Conclusion: 33

LIST OF FIGURES

Figure 3.1: Intercepting the product page: 5
Figure 3.2: Request with category=Gifts 6
Figure 3.3: Sending request to Repeater 6
Figure 3.4: Before adding Payload 7
Figure 3.5: Payload 'OR 1=1- - added 7
Figure 3.6: HTTP 200 OK response received 8
Figure 3.7: Lab solved 8
Figure 3.8: Login Bypass lab opened 10
Figure 3.9: Login Page 10
Figure 3.10: Adding username and password 11
Figure 3.11: Login request captured in HTTP history 11
Figure 3.12: Before Parameter modification 12
Figure 3.13: Username modified to administrator'- - 12
Figure 3.14: Response receiving HTTP 200 OK 13
Figure 3.15: Lab solved 13
Figure 3.16: Product category Pets request captured in HTTP history 15
Figure 3.17: Payload added 16
Figure 3.18: Payload checked increasing NULL value 16
Figure 3.19: Lab Solved 17
Figure 3.20: Lab opened in PortSwigger 18
Figure 3.21: Books entered in the search box 18
Figure 3.22: Result showing 0 results for Books 19
Figure 3.23: XSS Payload 19
Figure 3.24: Alert popup displaying 1 19
Figure 3.25: Lab Solved 20
Figure 3.26: Blog opened 21
Figure 3.27: XSS payload entered 22
Figure 3.28: Lab Solved 22
Figure 3.29: Invalid username and password 24

Figure 3.30: Login request in HTTP history 24
Figure 3.31: Username set as payload positio 25
Figure 3.32: Sniper attack with candidate username 25
Figure 3.33: Capturing Response Length 26
Figure 3.34: Valid username identified 26
Figure 3.35: Candidate password added 27
Figure 3.36: Comparing HTTP status codes 27
Figure 3.37:Correct password found 302 response 28
Figure 3.38: Lab solved 28
Figure 3.39: Logged in with wiener's credentials 30
Figure 3.40: 2FA code viewed in client email 30
Figure 3.41: URL changed to my-account 31
Figure 3.42: Lab solved 31

CHAPTER 1: WEB APPLICATION SECURITY

1.1 Introduction

Web application security is the practice of protecting web applications from vulnerabilities that can lead to unauthorized access, data exposure, or misuse.

This practical focuses on identifying common web application vulnerabilities using the PortSwigger Web Security Academy. It covers three major vulnerability categories: SQL injection, cross-site scripting (XSS), and authentication weaknesses.

The main objective was to understand how SQL Injection, XSS, and authentication weaknesses work, practice exploiting them in a controlled environment, and learn how they can be prevented.

1.2 Objective

The major objectives are:

- To identify SQL injection vulnerabilities.
- To understand how SQL injection can expose hidden application data.
- To demonstrate authentication bypass through SQL injection.
- To determine the number of columns returned by an injectable SQL query.
- To identify reflected and stored XSS vulnerabilities.
- To demonstrate a simple two-factor authentication bypass.

CHAPTER 2: Overview of Security Concepts

2.1 SQL Injection

SQL injection is a cyber-attack where malicious code is inserted into input fields on a website or app so an attacker can run commands on the backend database.

Types of SQL injection

i. **In-band SQLi:** In-band SQL Injection is a type of SQL injection where the attacker uses the same communication channel to send the SQL injection and receive the results from the database. It is of two types:

- **UNION-based SQLi:** Uses the UNION operator to retrieve data from other database queries.
- **Error-based SQLi:** Uses database error messages to obtain information about the database.

ii. **Blind SQLi:** Blind SQL Injection is a type of SQL injection where the application does not directly display the results or database errors from the injected SQL query. Instead, the attacker determines information by observing changes in the application's response, behavior, or timing**.** Its types are:

- **Boolean-based SQLi:** Determines information by comparing different responses when a condition is true or false.
- **Time-based SQLi:** Determines information by observing differences in the application's response time.

iii. **Out-of-Band SQLi:** It is a type of SQL injection where the attacker sends the injection through the web application but receives the database information through a separate channel**.**

2.2 Cross-Site Scripting (XSS)

Cross-Site Scripting is a web security vulnerability that allows an attacker to inject malicious client-side scripts into a trusted website. These scripts execute in the victim's browser and run with the same permissions as the web application.

Types of Cross-site Scripting

i. Reflected XSS: It occurs when malicious user input is immediately reflected back by the web server in the response. The payload is not permanently stored in the application.

- **How it works:**
  - Attacker creates a malicious URL/request.
  - Victim opens or submits it.
  - The application reflects the input into the webpage.
  - The victim's browser interprets the injected content as script.

ii. Stored XSS: It occurs when malicious input is stored permanently by a web application, commonly in a database. The XSS payload remains stored until it is removed or changed.

- **How it works:**
  - Attacker submits malicious input.
  - The application stores the input.
  - A user visits the affected page.
  - The stored payload is delivered to the user's browser.
  - The browser executes the injected script.

iii. DOM-Based XSS: It occurs when JavaScript takes something provided by the user and puts it directly into a webpage without checking if it is safe.

- **How it works:**
  - Attacker provides malicious input.
  - JavaScript reads the user-controlled input.
  - The input is inserted into the webpage without proper sanitization or encoding.
  - The browser interprets the injected content as executable code.

2.3 Authentication

Authentication is the process of confirming the identity of a user, device, or system by validating provided credentials before granting access to a network or its resources. It maintains security, reliability, and trust within the system.

- **Types of Authentications:**

i. **Single Factor Authentication (SFA**): It uses only one method to verify a user's identity usually a username and password. If the entered password is correct, the user gets access.

ii. **Two Factor Authentication (2FA):** It uses two different methods to verify a user's identity and usually combines password, phone OTP, or a security token. It is more secure than Single Factor Authentication (SFA).

iii. **Multi Factor Authentication (MFA):** It uses two or more different methods to verify a user's identity. It can be password, OTP code, security token or fingerprint, etc. Even if one factor is compromised, the other factors provide additional protection. It provides strong security as compared to both Single Factor Authentication and Two Factor Authentication.

CHAPTER 3: Practical Task

3.1 Lab 1: SQL Injection (Hidden Data)

1. **Vulnerability:** SQL Injection in the product category filter
2. **Objective:** To perform a SQL injection attack that causes the application to display one or more unreleased products.
3. **Steps Performed:**

Step 1: Opened the SQL Injection – Hidden Data lab in Burp Suite's browser.

Step 2: Turned Intercept ON and opened the product page.

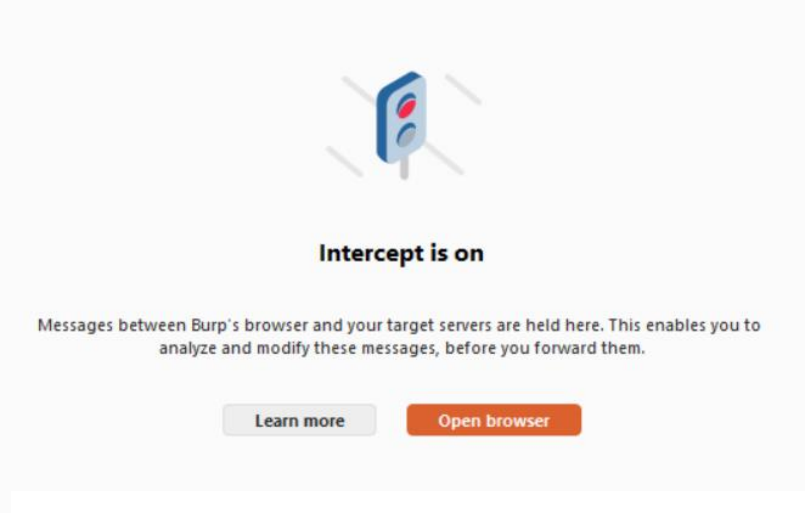


Figure 3.1: Intercepting the product page:

Step 3: Clicked Gifts and intercepted the request containing category=Gifts.

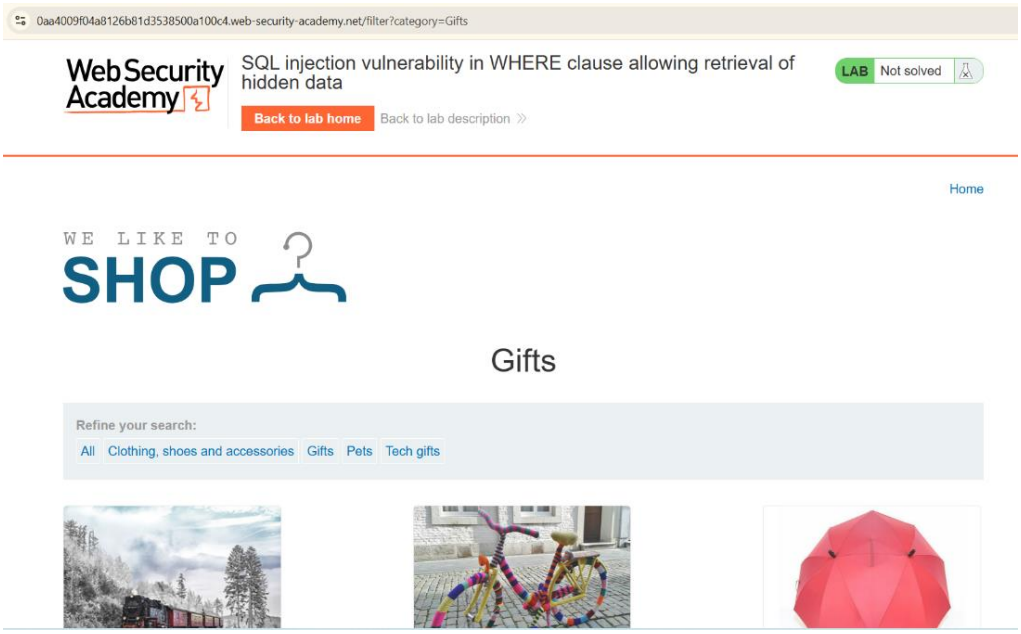


Figure 3.2: Request with category=Gifts

Step 4: Check the Http history and sent the request to Repeater.

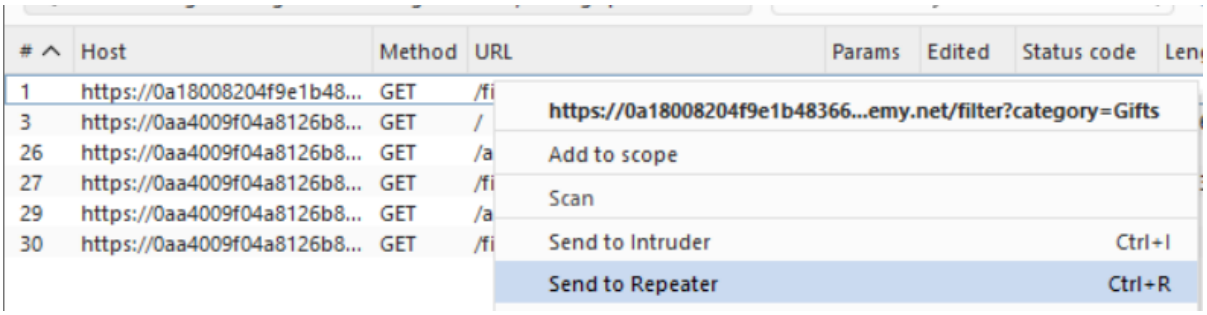


Figure 3.3: Sending request to Repeater

Step 5: In repeater request, category=Gifts are then added with ' OR 1=1--.

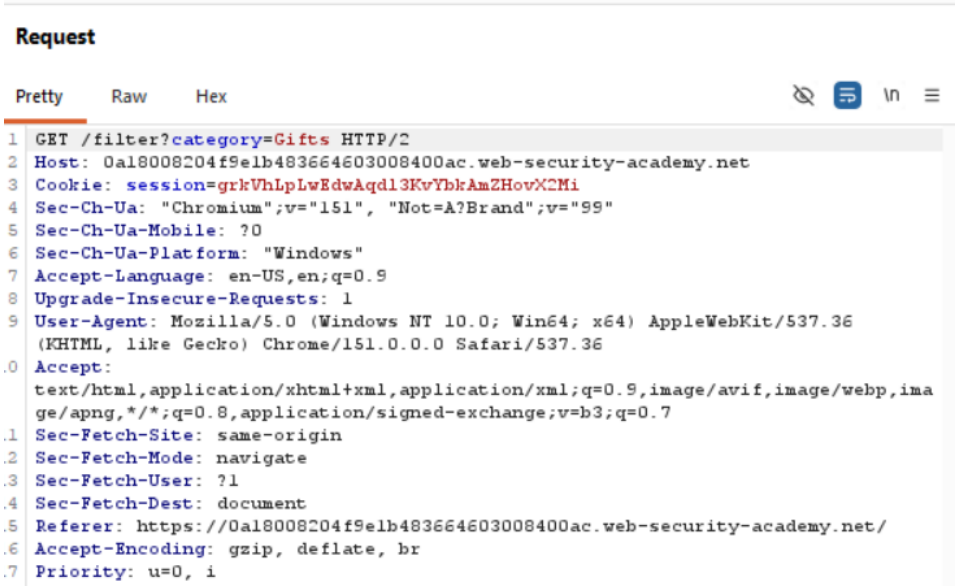


Figure 3.4: Before adding Payload



Figure 3.5: Payload 'OR 1=1- - added

Step 6: The modified request was sent and received HTTP 200 OK.

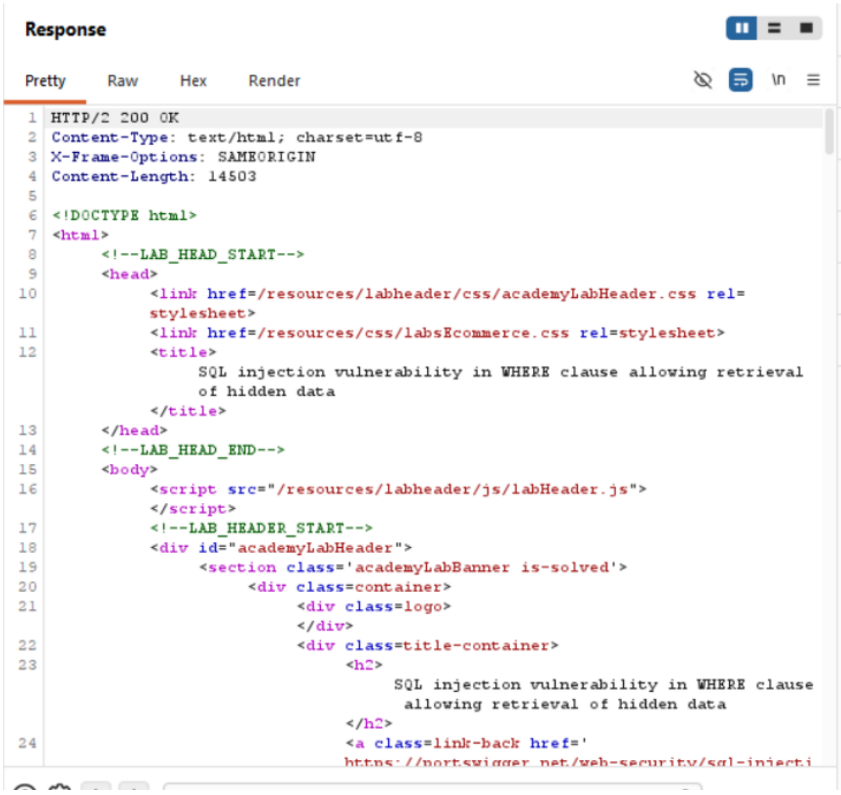


Figure 3.6: HTTP 200 OK response received

Step 7: Then the manipulated response displayed the products, and the lab was marked solved**.**

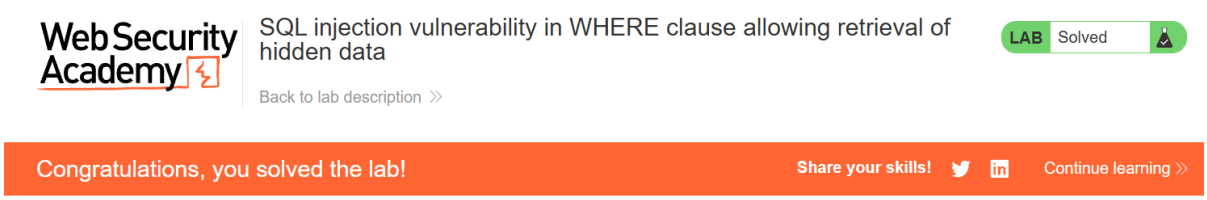


Figure 3.7: Lab solved

4. **Payload Used:** ‘OR 1=1--

This payload was entered into the category parameter to test the application was vulnerable to SQL injection. The OR 1=1 condition is always true, while –comments out the remaining part of the SQL query.

5. **Impact:**

An attacker can use SQL Injection to change the database query and see information that should be hidden. This may lead to unauthorized access to sensitive data.

6. **Mitigation:**

The application should use prepared statements or parameterized queries instead of directly adding user input to SQL queries. Input validation should also be used to prevent malicious SQL commands.

3.2 Lab 2: SQL Injection (Login Bypass)

1. **Vulnerability:** SQL Injection in the login page.
2. **Objective:** To perform a SQL injection attack that logs into the application as the administrator user.
3. **Steps Performed:**

Step 1: In Burp Suite's built-in browser, login Bypass lab was opened

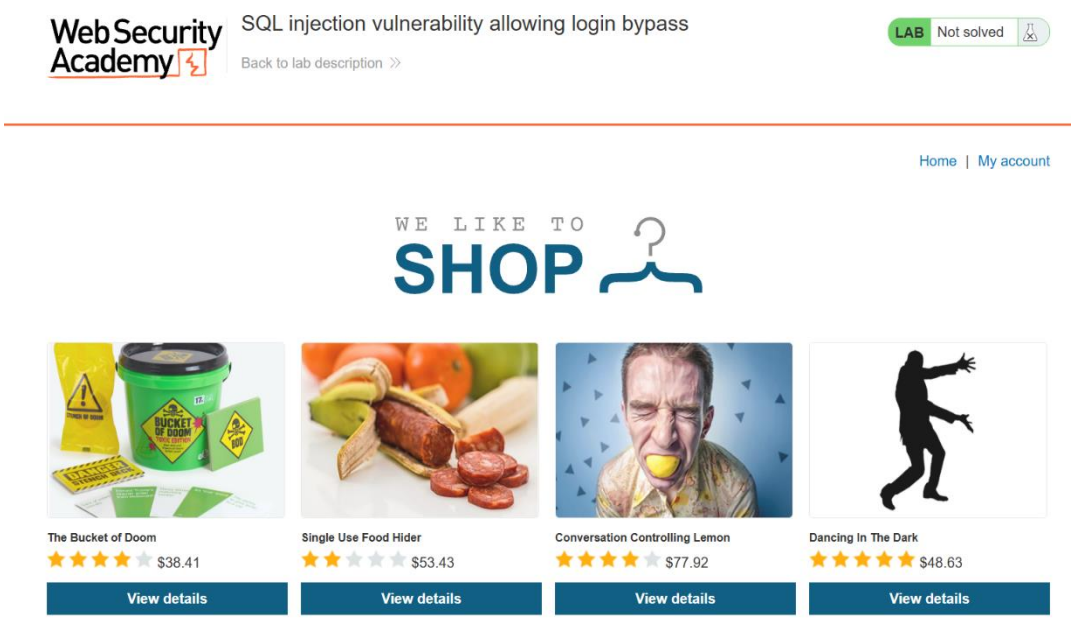


Figure 3.8: Login Bypass lab opened

Step 2: Then click My account, a login page will be opened.

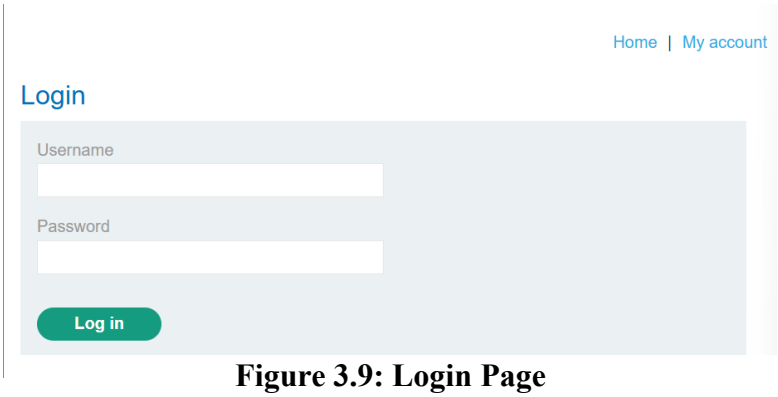


Figure 3.9: Login Page

Step 3: After adding username and password, intercept was turned ON.

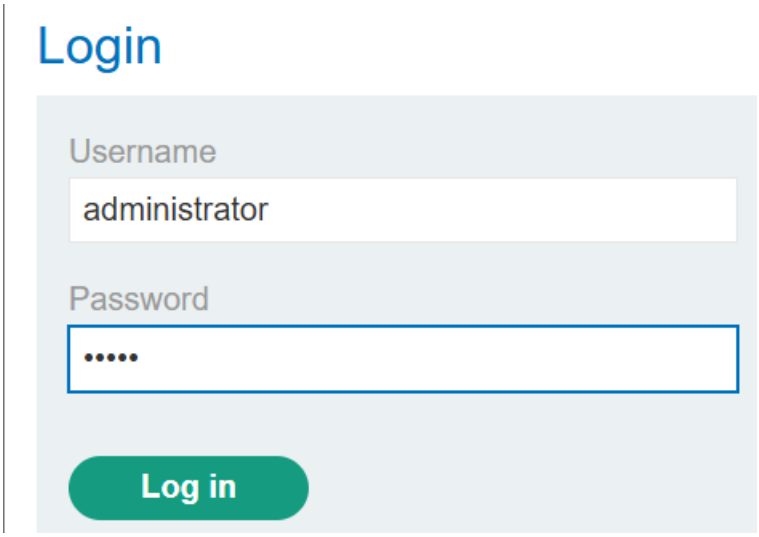


Figure 3.10: Adding username and password

Step 4: In HTTP history the login request was found and login request was sent to Repeater.

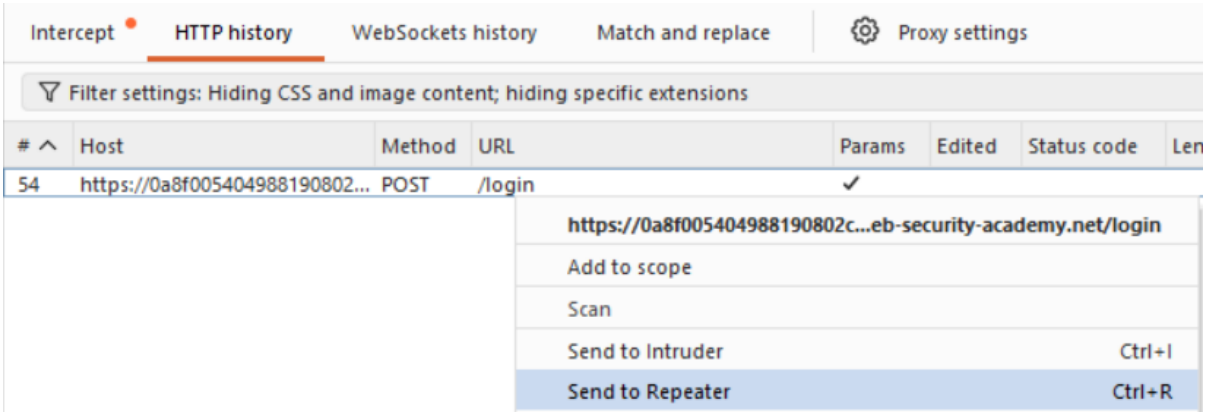


Figure 3.11: Login request captured in HTTP history

Step 5: In the Repeater request, username=administrator was changed to username=administrator'--

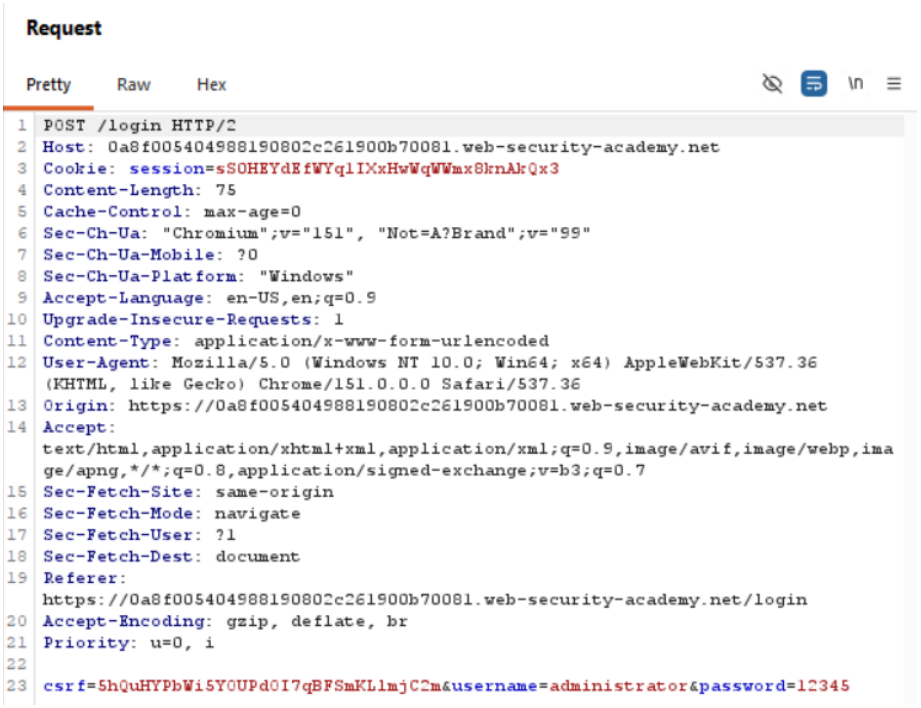


Figure 3.12: Before Parameter modification

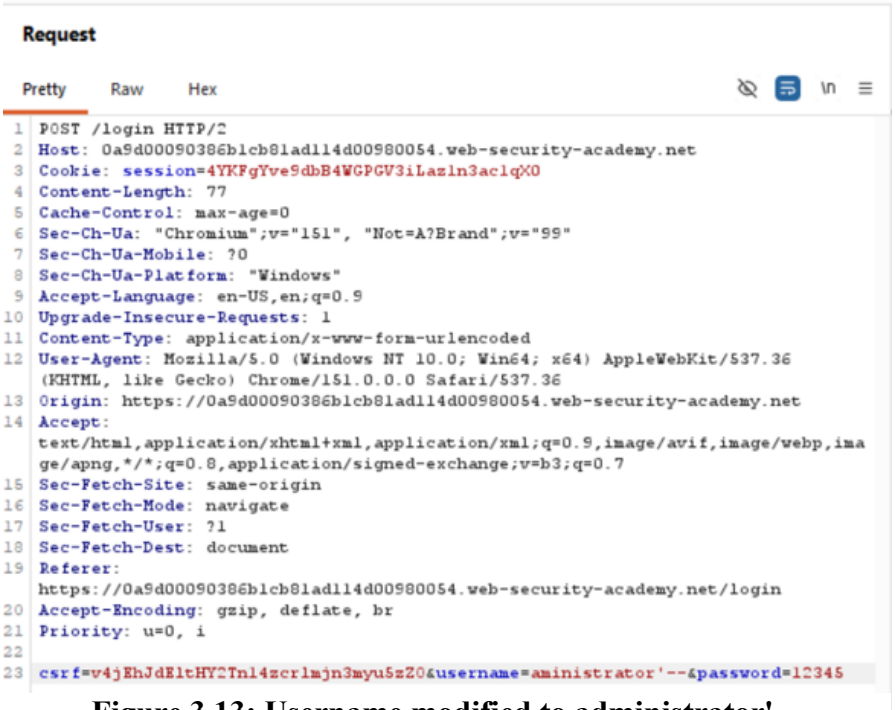


Figure 3.13: Username modified to administrator'- -

Step 6: The modified request was sent and received HTTP 200 OK.

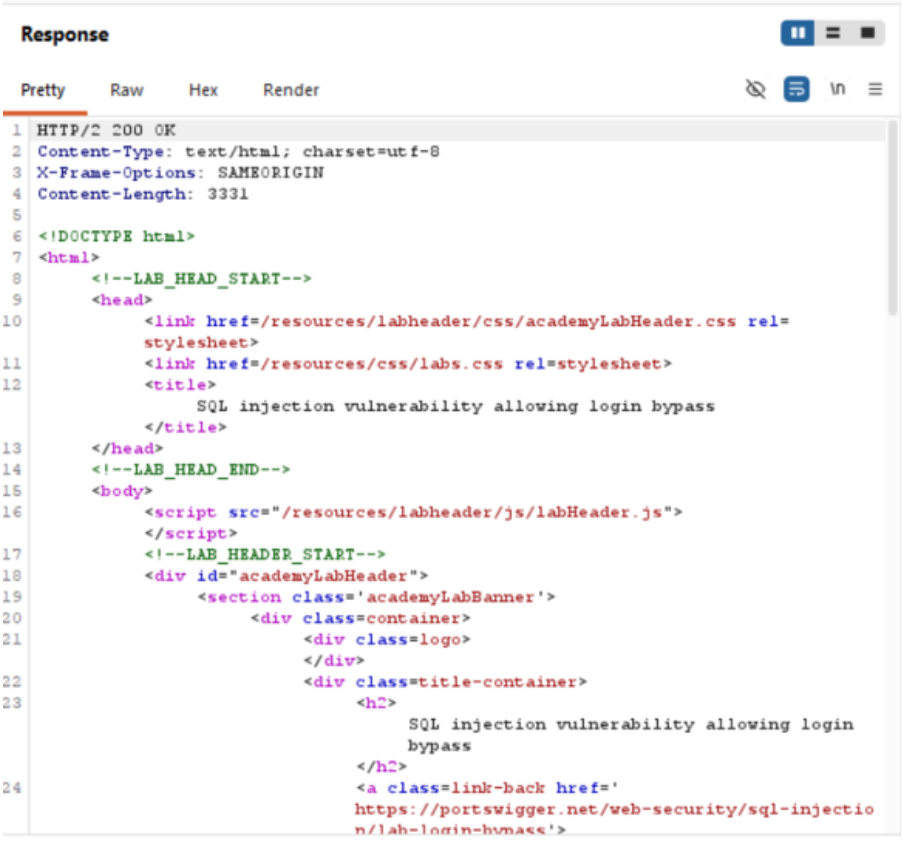


Figure 3.14: Response receiving HTTP 200 OK

Step 7: Then the modified request bypassed the login, and the application logged in as the administrator user. The lab was marked solved.

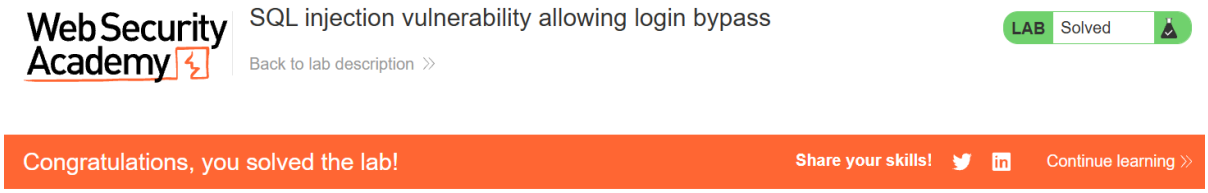


Figure 3.15: Lab solved

4. **Payload Used:** administrator ‘- -

The payload administrator’—bypassed the password check by commenting out the remaining part of the SQL query. This allowed the application to login as the administrator without requiring the correct password.

5. **Impact:**

An attacker can bypass the login and access an account without knowing the correct password. This can lead to unauthorized access to user accounts.

6. **Mitigation:**

Use prepared statements or parameterized queries for login inputs. Validate user input and never build SQL queries by directly combining usernames or passwords with SQL statements.

3.3 Lab 3: SQL Injection (Union Attack)

1. **Vulnerability:** SQL injection in the product category filter.
2. **Objective:** To determine the number of columns returned by the query by performing a SQL injection UNION attack that returns an additional row containing null values.
3. **Steps Performed:**

Step 1: SQL Injection UNION Attack lab using Burp Suite's built-in browser.

Step 2: Intercept was turned ON and product category Pets was clicked.

Step 3: HTTP history was opened and the request containing category=Pets was sent to Repeater.

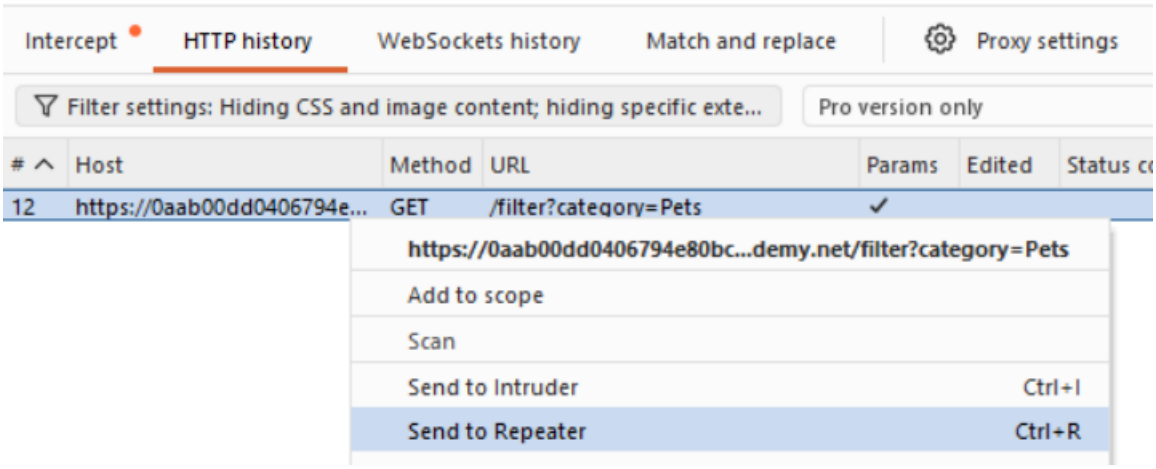


Figure 3.16: Product category Pets request captured in HTTP history

Step 4: In Repeater, tested the category parameter with a UNION payload containing NULL values.

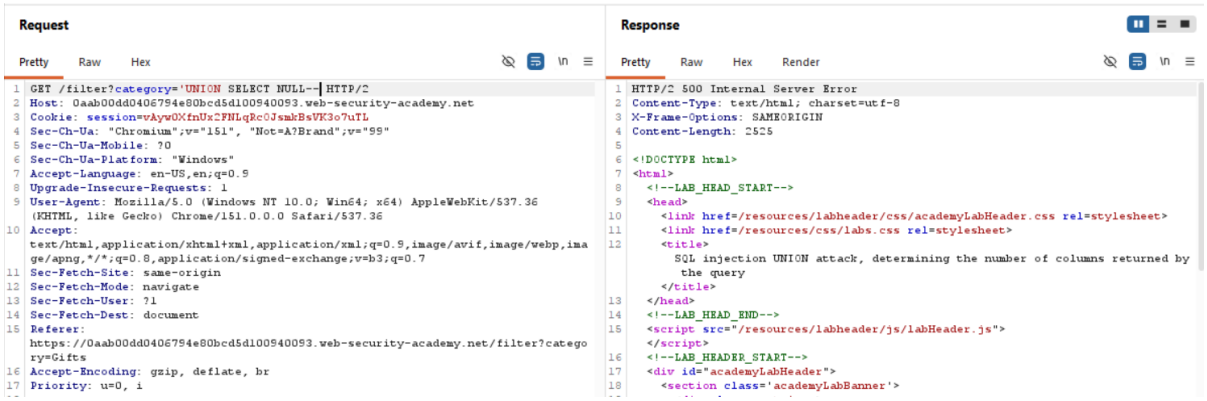


Figure 3.17: Payload added

Step 5: Increased the number of NULL values until the request returned successfully.

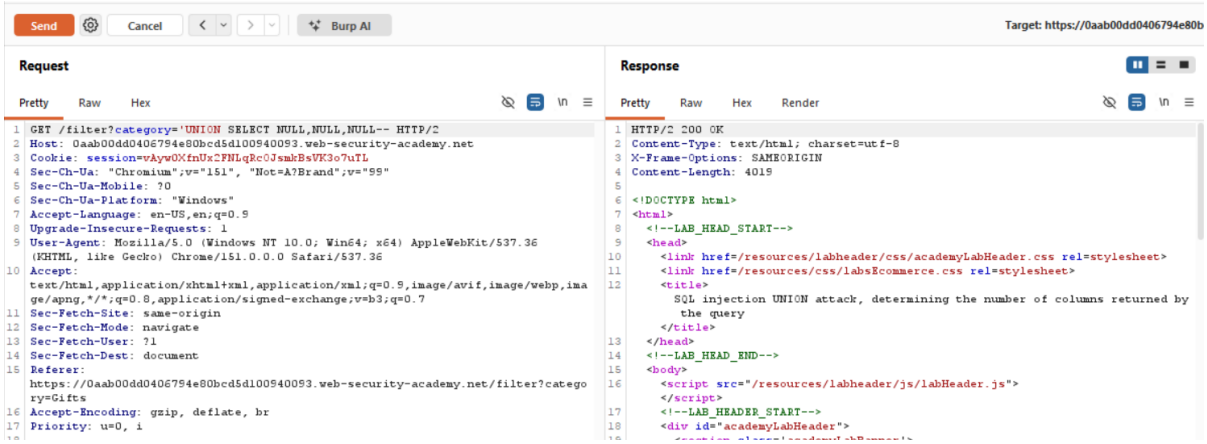


Figure 3.18: Payload checked increasing NULL value

Step 6: The successful response showed the correct number of columns returned by the query.

Step 7: The lab was marked solved.



Figure 3.19: Lab Solved

4. **Payload Used:** 'UNION SELECT NULL- -

The number of NULL values was increased one by one until a successful response was received, which helped to determine the number of columns.

5. Impact:

An attacker can use the SQL injection vulnerability to access additional data from the database.

6. Mitigation:

Use prepared statements or parameterized queries to prevent attackers from changing SQL queries.

3.4 Lab 4: Cross-Site Scripting (Reflected XSS)

1. **Vulnerability:**
2. **Objective:** To perform cross-site scripting attack that calls the alert function.
3. **Steps Performed:**

Step 1: Reflected XSS lab was opened in PortSwigger.

Step 2: Find the search box.



Figure 3.20: Lab opened in PortSwigger

Step 3: Books was typed in search box and search button was clicked.

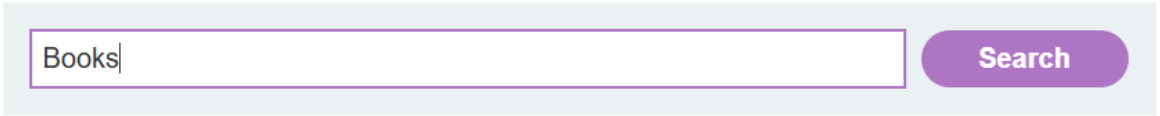


Figure 3.21: Books entered in the search box

Step 4: The result page shows 0 results for ‘Books’.

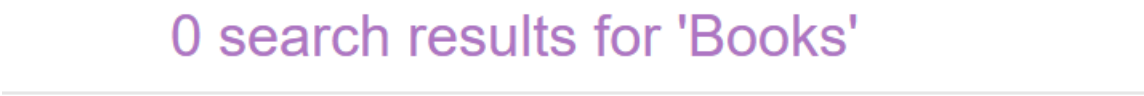


Figure 3.22: Result showing 0 results for Books

Step 5: To test whether the search input is vulnerable, a payload was used.

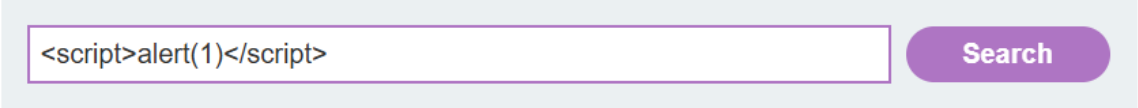


Figure 3.23: XSS Payload

Step 6: An alert popup showing 1 appears, the XSS has successfully executed.

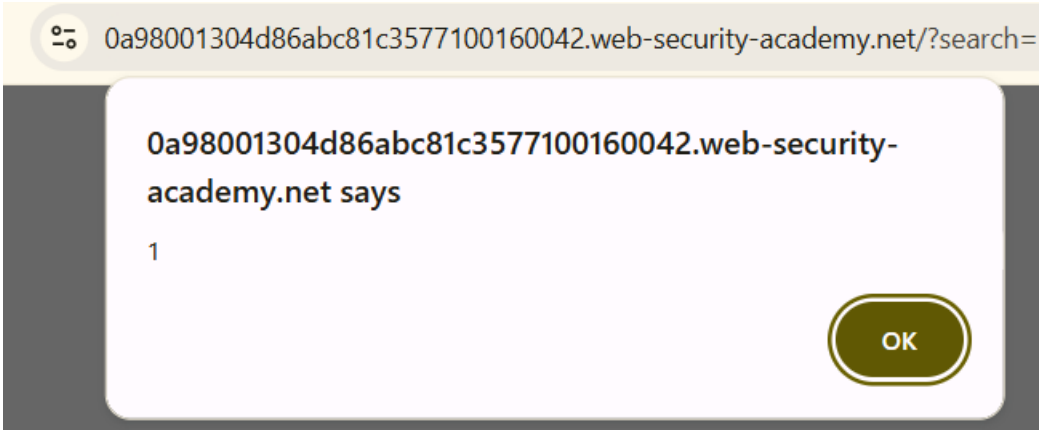


Figure 3.24: Alert popup displaying 1

Step 7: The lab was marked solved.

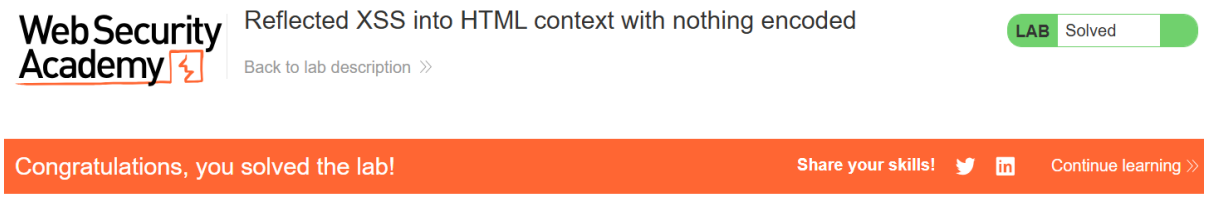


Figure 3.25: Lab Solved

4. **Payload Used:** <script>alert (1) </script>

The <script> tag tells the browser to execute JavaScript, and alert (1) displays a popup message. The popup confirms that the injected JavaScript was executed.

5. Impact:

An attacker can execute JavaScript in a victim's browser. This may allow the attacker to modify webpage content or perform actions on behalf of the victim.

6. Mitigation:

Encode user input before displaying it on a web page so that it is treated as text instead of JavaScript also validate the input.

3.5 Lab 5: Cross-Site Scripting (Stored XSS)

1. **Vulnerability:** Stored Cross-Site Scripting (XSS) in the comment functionality.
2. **Objective:** To submit a comment that calls the alert function when the blog post is viewed.
3. **Steps Performed:**

Step 1: The Stored XSS was opened in the browser.

Step 2: Then go through the blog and view post was clicked.

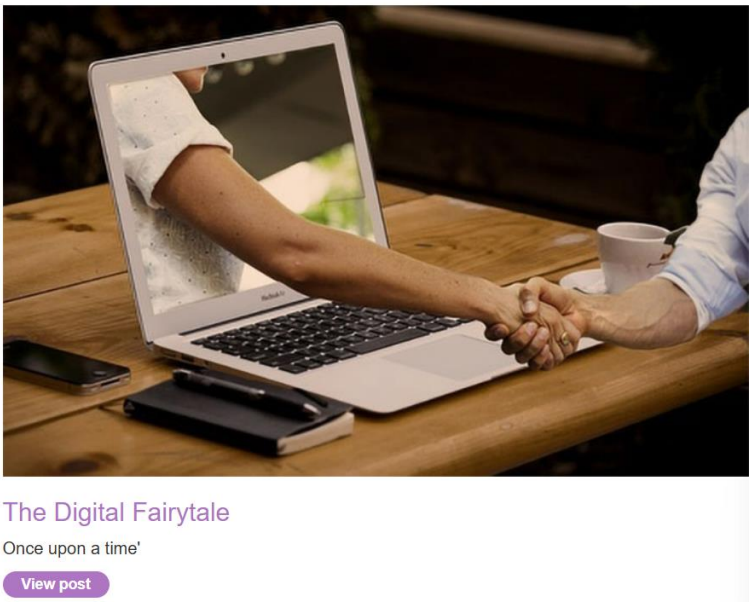


Figure 3.26: Blog opened

Step 3: Scrolled down to the comment section and a payload <script>alert (1) </script> was entered in the comment field and comment was submitted.

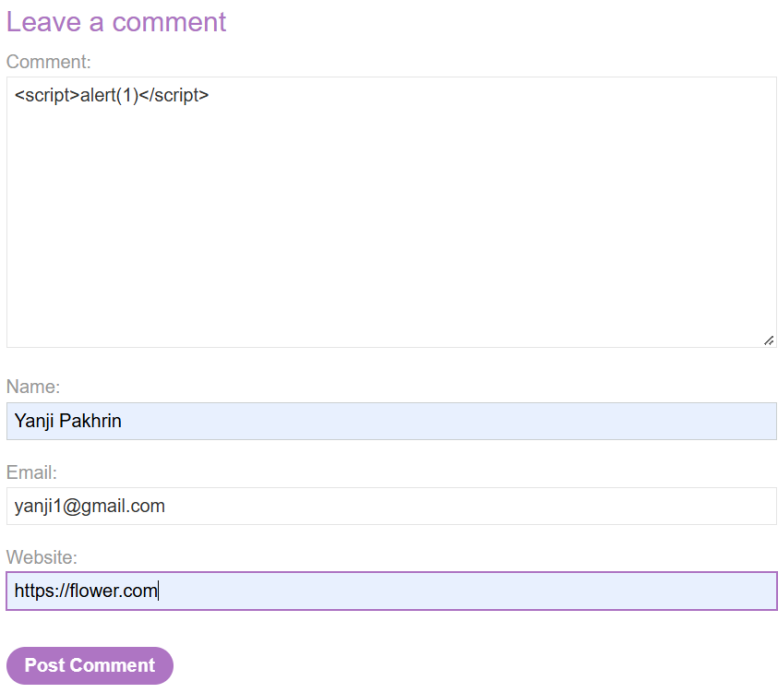


Figure 3.27: XSS payload entered

Step 4: The lab was marked solved

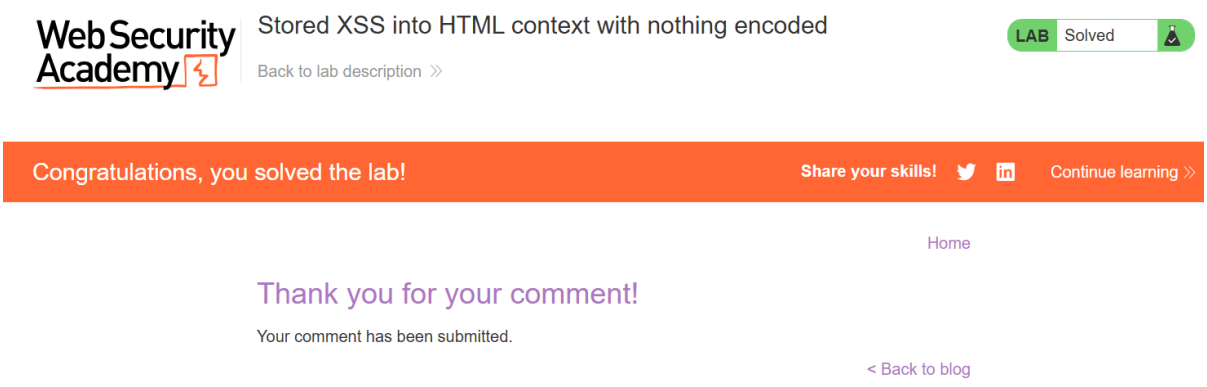


Figure 3.28: Lab Solved

4. **Payload Used:** <script>alert (1) </script>

The payload was submitted through the comment field. The JavaScript was stored by the application and executed when the blog post was viewed.

5. Impact:

An attacker can store malicious JavaScript in a comment, which may execute in the browsers of users who view the affected blog post. This can be used to modify webpage content or perform actions on behalf of the victim.

6. Mitigation:

Encode user input before displaying comments on the webpage so that it is treated as text instead of JavaScript.

3.6 Lab 6: User Enumeration

1. **Vulnerability:**
2. **Objective:** To enumerate a valid username, brute-force this user's password, then access their account page.
3. **Steps Performed:**

Step 1: Login page was opened in Burp Suite.

Step 2: An invalid username and password was entered and Intercept turned ON and the login form was submitted.

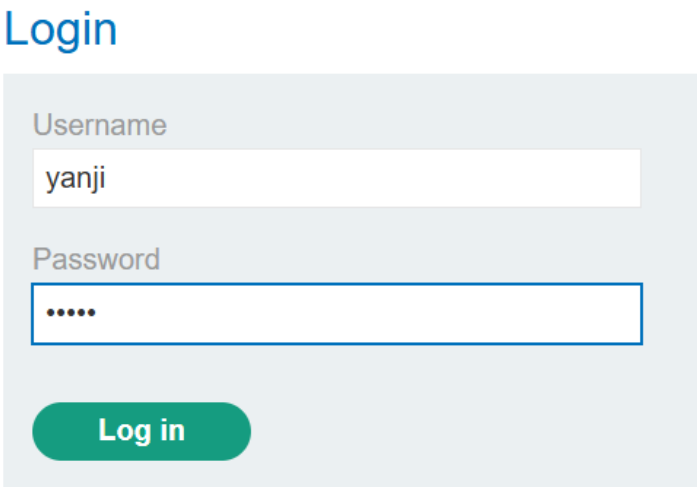


Figure 3.29: Invalid username and password

Step 3: Then in HTTPS history login request was found and it was sent to intruder.

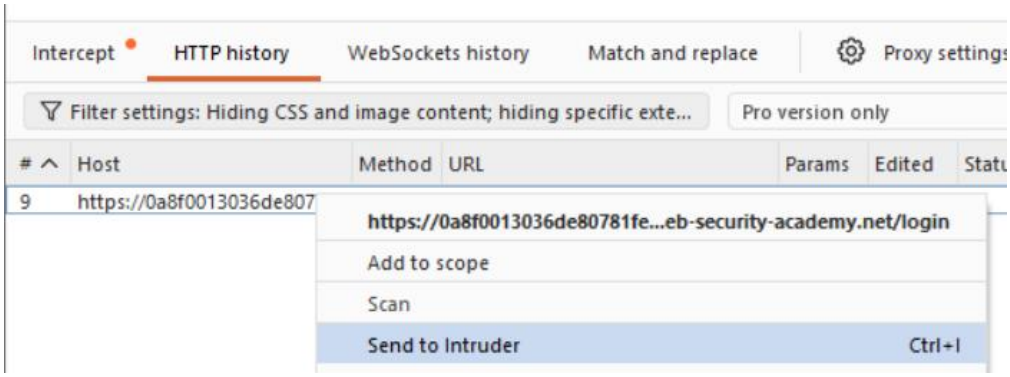


Figure 3.30: Login request in HTTP history

Step 4: Username parameter was selected and add § to set it as the payload position in Burp Intruder

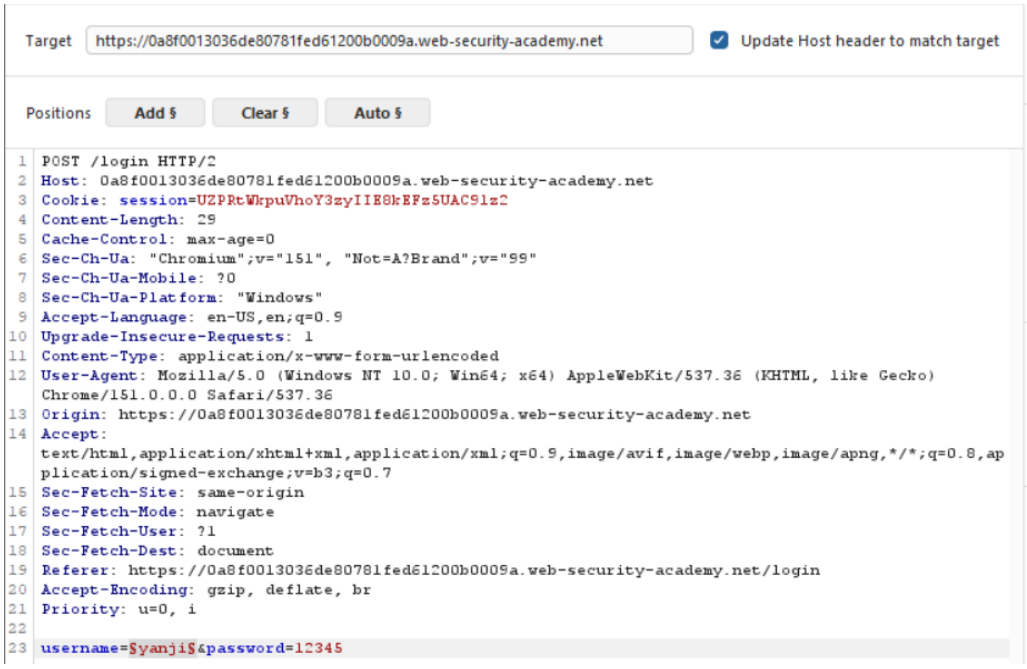


Figure 3.31: Username set as payload positio

Step 5: Snipper was selected as the attack type and Simple list as payload type was selected and candidate usernames were added. Then click Start attack.

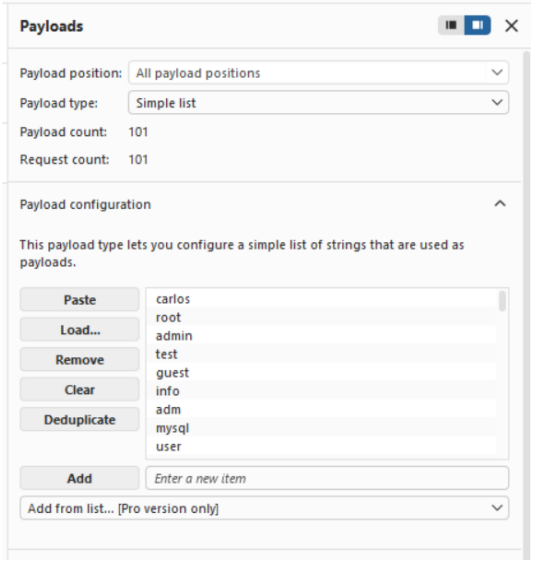


Figure 3.32: Sniper attack with candidate username

Step 6: Compared the response lengths and identified the valid username from the different response.

Request	Payload	Status code	Response received	Error	Timeout	Length
95	att	200	229			3352
96	au	200	227			3352
97	auction	200	286			3352
98	austin	200	183			3352
99	auth	200	247			3352
100	auto	200	236			3352
101	autodiscover	200	199			3352
78	appserver	200	223			3354

Figure 3.33: Capturing Response Length

Step 7: The valid username form the Payload column was noted and replaced the username payload with the identified username.

Positions | Add § | Clear § | Auto §

```
1  POST /login HTTP/2
2  Host: 0a8f0013036de80781fed61200b0009a.web-security-academy.net
3  Cookie: session=UZPRtWkpuVhoY3zyIIE8kEFz5UAC9lz2
4  Content-Length: 29
5  Cache-Control: max-age=0
6  Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7  Sec-Ch-Ua-Mobile: ?0
8  Sec-Ch-Ua-Platform: "Windows"
9  Accept-Language: en-US,en;q=0.9
10 Upgrade-Insecure-Requests: 1
11 Content-Type: application/x-www-form-urlencoded
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
13 Origin: https://0a8f0013036de80781fed61200b0009a.web-security-academy.net
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0a8f0013036de80781fed61200b0009a.web-security-academy.net/login
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 username=appserver&password=$12345$
```

Figure 3.34: Valid username identified

Step 8: The password payload type was selected Simple list and the candidate passwords were added.

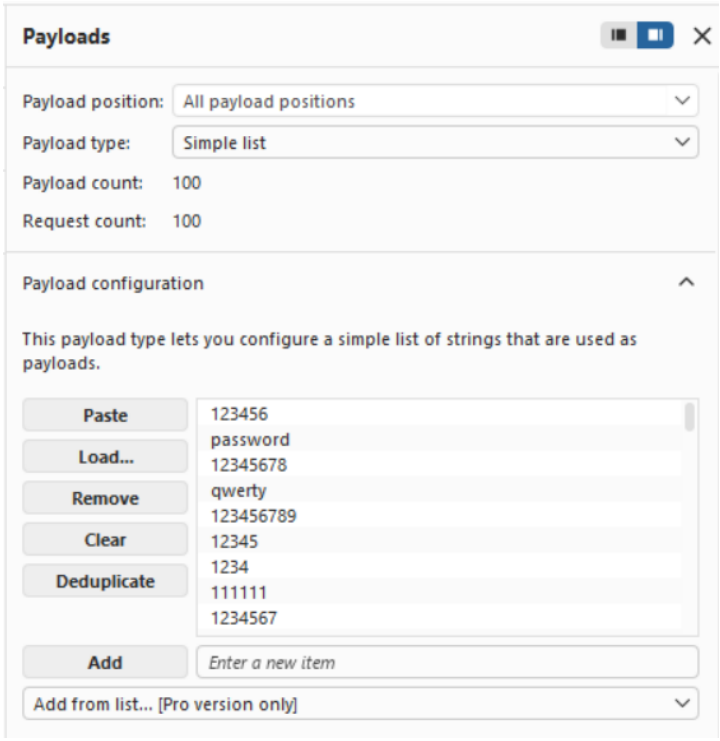


Figure 3.35: Candidate password added

Step 9: Start attack was clicked and HTTP status codes was compared and correct password was identified from the request with a 302-status code.

Request	Payload	Status code	Response received	Error	Timeout	Length
70	pass	200	288			3354
71	maggie	200	285			3354
72	159753	200	247			3354
73	aaaaaa	302	187			191
74	ginger	200	211			3441
75	princess	200	251			3441
76	joshua	200	187			3441

Figure 3.36: Comparing HTTP status codes

Step 10: Logged in using the identified username and password and accessed the user account page.

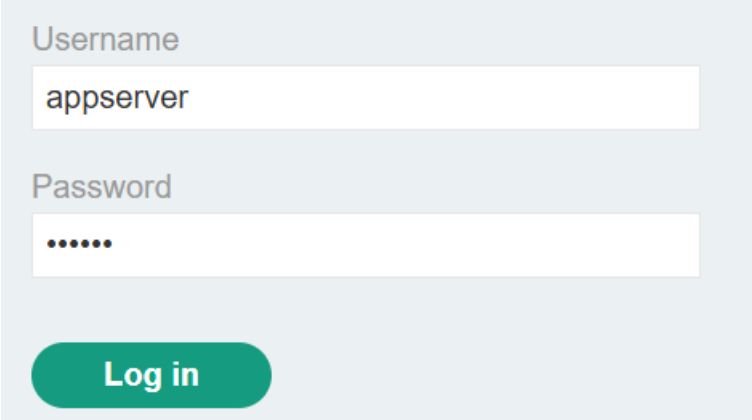


Figure 3.37:Correct password found 302 response

Step 11: The lab was marked solved.



Figure 3.38: Lab solved

4. Payload Used:

Candidate usernames and candidate passwords provided by PortSwigger were used as payloads in Burp Intruder.

5. Impact:

An attacker can identify valid usernames and repeatedly test passwords to gain unauthorized access to user accounts. This may lead to account compromise and unauthorized access to sensitive information.

6. Mitigation:

Implement rate limiting and account lockout mechanisms to prevent repeated password attempts. Multi-factor authentication (MFA) should also be used for additional protection.

3.7 Lab 7: 2FA Simple Bypass

1. **Vulnerability:** 2FA bypass via URL manipulation
2. **Objective:** To bypass 2FA and access Carlos's account page.
3. **Steps Performed:**

Step 1: 2FA Simple Bypass lab was opened in the browser.

Step 2: Logged in using the provided credentials wiener: peter.

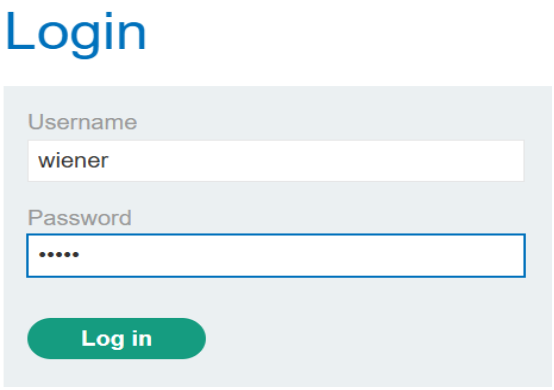


Figure 3.39: Logged in with wiener's credentials

Step 3: Email client was clicked and 2FA code was viewed and the code was entered, then account page was accessed and the /my-account URL was noted.

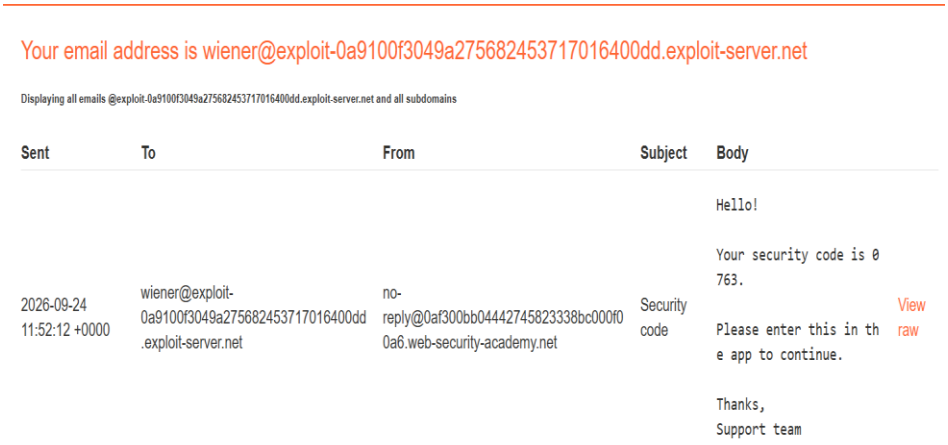


Figure 3.40: 2FA code viewed in client email

Step 4: Logged out of the account.

Step 5: Logged in using Carlos's credentials carlos: montoya.

Step 6: Reached the 2FA verification page.

Step 7: The URL was changed to /my-account without entering the 2FA code.

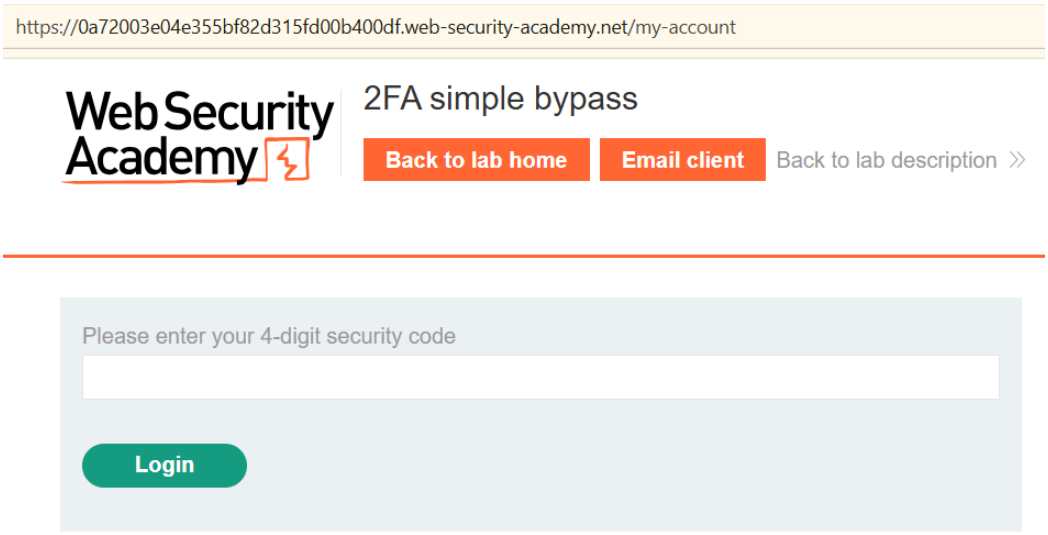


Figure 3.41: URL changed to my-account

Step 8: Carlos's account page was accessed successfully.

Step 9: The lab was marked solved.

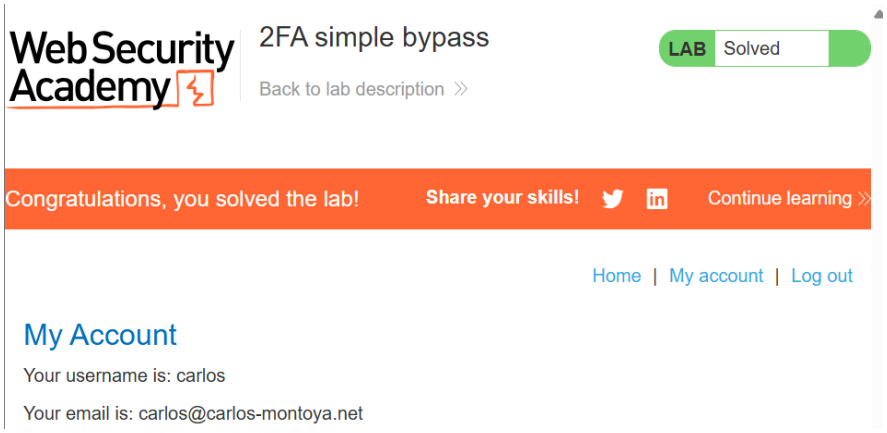


Figure 3.42: Lab solved

4. **Payload:** No payload used

5. Impact:

An attacker who obtains a user's username and password may bypass the 2FA verification step and access the user's account. This can lead to unauthorized access to account information.

6. Mitigation:

The application should verify the 2FA code before granting access to protected account pages. Server-side access controls should ensure that users cannot bypass 2FA by directly changing the URL.

3.8 Challenges

- At first understanding how Burp Suite works and how to use its different features was challenging.

3.9 Key Learnings

- SQL Injection can be used to manipulate database queries.
- UNION attacks can help determine the number of columns in a query.
- Learned how password brute-force attacks can be performed using Burp Intruder.
- Learned how Reflected and Stored XSS can execute JavaScript in a browser.
- Learned how proper input validation, parameterized queries, output encoding, and strong authentication controls help prevent these vulnerabilities.

3.10 Conclusion:

Overall, this lab helped to develop a understanding of common web vulnerabilities and how they can be tested using PortSwigger Web Security Academy. It also provided hands-on experience with Burp Suite and web security techniques.